不同优先队列下 Dijkstra 算法的实现与评测

匿名报告底稿

2026 年 10 月 28 日

摘要

本项目在统一的 CSR 图表示与优先队列接口下，实现并比较七种 Dijkstra 队列策略，包括必须考察的 Fibonacci 堆；另外加入 FIFO SPFA 和 SLF+LLL SPFA 作为参照。实验覆盖随机图的 171 组配置、固定密度复杂度实验的 126 组配置，以及 NY、BAY 两个 DIMACS 路网各 1000 条查询、每图九种实现。本次 WSL2 Release 测量中，路网上最好的 Dijkstra 队列为 radix heap，NY、BAY 的平均墙钟时间分别为 12.621 和 13.873 ms/查询；SLF+LLL SPFA 分别为 7.267 和 9.258 ms/查询。Fibonacci 堆没有在这两张路网上显示实际速度优势。本文同时区分渐近保证、有限规模拟合与实际运行时间，讨论整数键范围、输入可达性及重复次数对结论的限制。

关键词：Dijkstra；Fibonacci 堆；radix heap；DIMACS；复杂度分析

底稿说明。正文及图表已填入本次实测数据，数字表由脚本生成。作者分工、声明签名和源码注释比例仍需项目成员核对。默认匿名开关开启；正式提交前请按当前课程通知确认匿名与命名要求。

目录

第 1 章 引言 **3**

1.1 问题与目标 3

1.2 实现与实验范围 3

1.3 本轮结果的解释边界 3

第 2 章 数据结构与算法说明 **4**

2.1 CSR 图与共享接口 4

2.2 Dijkstra 的主循环与正确性 4

2.3 二叉堆、段树与 PBDS 4

2.4 Fibonacci 堆 5

2.5 单调 radix heap 5

2.6 两种标签修正参照算法 6

第 3 章 测试结果 **7**

3.1 环境与计时口径 7

3.2 正确性用例表 7

3.3 USA 路网输入与结果 9

3.4 随机图参数扫描 10

3.5 固定密度实验与拟合 11

第 4 章 分析与讨论 **13**

4.1 时间与空间复杂度 13

4.2 为何 Fibonacci 堆没有最快 13

4.3 Radix 与 SPFA 结果的意义 14

4.4 模型拟合与测量限制 14

4.5 后续改进与底稿结论 14

附录 A 源代码与复现说明 **15**

A.1 源码组织 15

A.2 数据与作图复现 16

参考文献 **18**

作者分工 **19**

声明 **19**

签名 **19**

第 1 章 引言

1.1 问题与目标

给定有向图 $G=(V,E)$、非负边权 $w:E\to\mathbb{Z}_{\geq 0}$ 和源点 s，求从 s 到各顶点的最短距离 $d[v]$。不可达顶点的距离记作 ∞。记 $n=|V|$、$m=|E|$；本文的 m 始终指有向弧条数。课程项目要求使用至少两种堆结构且包含 Fibonacci 堆，在 USA 路网中至少评测 1000 对查询，并给出运行时间、规模曲线与分析。

Dijkstra 的贪心顺序依赖最小优先队列。二叉堆、段树及 Fibonacci 堆对插入、减键和取最小元素的处理方式不同，因而具有不同的渐近代价和实际常数。对非负整数距离，还可以使用单调的 radix heap。本报告关注两个问题：理论上更便宜的减键操作能否转化为实际收益；固定图结构和输入后，不同实现的速度次序如何变化。

1.2 实现与实验范围

七种 Dijkstra 队列为标准库优先队列、手写二叉堆、段树堆、Fibonacci 堆、radix heap、GNU PBDS pairing heap 和 thin heap。除标准库版本以重复插入模拟减键外，其他版本支持按顶点定位并更新键。FIFO SPFA 与 SLF+LLL SPFA 是基于队列的标签修正算法，不属于 Dijkstra 的堆变体，在报告中作为独立参照。

所有实现共用 `general-dijkstra.hpp` 中的算法主循环以及 CSR 图。随机图与固定密度实验对同组参数使用固定种子 42 和同一个生成规则。路网采用 `USA-road-d.NY` 与 `USA-road-d.BAY`，查询文件是项目生成的 1000 对随机查询，种子为 971；查询文件不是官方原始下载。本轮未测试 NE、E 或全美 USA 图，结论不外推到这些规模。

1.3 本轮结果的解释边界

尽管路网输入格式为点对点查询，现有算法会从每个源点计算完整距离数组，直到队列为空，目标点仅用于取校验和。因此本文的“ms/查询”是**一个点对触发一次完整单源搜索的平均时间**，不能等同于遇到目标点即停止的点对点搜索延迟。本轮不包含路径恢复、预处理索引或多线程并行搜索。

随机图和复杂度实验每组重复三轮；每个路网实现仅运行一个包含 1000 查询的批次。所有图表和表格都来自同一轮已校验的 CSV 快照，避免把旧报告中的测量值混入比较。

第 2 章 数据结构与算法说明

2.1 CSR 图与共享接口

顶点编号为 $1,\dots,n$。CSR 使用连续边数组和长度为 $n+1$ 的偏移数组，按起点将弧分组；顺序访问某顶点的邻接弧需要与该顶点出度成正比的时间。边保存终点与整数权重，最短距离使用 64 位有符号整数。距离数组的初始无穷大取该类型最大值的四分之一，为本轮输入的距离加法留出余量；该实现没有覆盖任意大输入的通用溢出检测。

优先队列对外提供 `push_or_decrease(key,v)`、`pop()`、`empty()` 和 `clear()`。支持减键的实现通过顶点索引定位元素，共享基类用时间戳区分本轮在堆中的顶点。`pop()` 后清除对应在堆标记，完整搜索结束时队列必须为空并可复用。

2.2 Dijkstra 的主循环与正确性

非负权重保证：取出的最小有效标签不会再被后来经过未处理顶点的路径改进。初始时源点距离为零；每次松弛只用已发现路径的长度更新标签，因而有限标签始终对应某条实际路径。结合最小有效标签的定型性质，归纳可得搜索结束后所有可达顶点距离正确。标准库版本可能含有旧键，故主循环统一执行过期条目检查。

```
DIJKSTRA(G, s, Q):
  fill d[0..n] with INF
  d[s] = 0; Q.push_or_decrease(0, s)
  while not Q.empty():
    (key, u) = Q.pop()
    if key > d[u]: continue
    for each (v, w) in G[u]:
      if d[u] + w < d[v]:
        d[v] = d[u] + w
        Q.push_or_decrease(d[v], v)
  return d
```

Listing 1: 共享 Dijkstra 伪代码

2.3 二叉堆、段树与 PBDS

标准库优先队列。最小堆保存 (d,v)。每次改进距离直接插入新条目，取出时丢弃旧条目。因此堆中可能同时有同一顶点的多个键，空间随成功松弛次数增长。

手写二叉堆。数组 `heap[pos]` 保存顶点，`idx[v]` 保存位置，`key[v]` 保存当前键。插入把顶点放到末尾并上浮；减键从 `idx[v]` 上浮；取最小值把末尾移到根并下沉，每次交换同步更新两个顶点的位置。每个在堆顶点只有一个条目。

段树堆。每个顶点对应一个叶子，内部结点保存子区间的最小键及其顶点。插入或减键沿根路径更新；取最小值读取根的顶点，再将相应叶子置为 ∞。更新过程中即使最小值不变，只要对应顶点改变也必须继续传播，以避免相等键导致过期顶点；最大编号顶点与哨兵叶子必须分开。

PBDS pairing 与 thin heap。适配器保存每个顶点的 `point_iterator`，首次入堆调用 `push`，随后用 `modify` 更新条目。内部堆序维护交给 GNU 库。本项目没有重新实现两种库堆，也不把它们的减键摊还界简单等同于 Fibonacci 堆。

2.4 Fibonacci 堆

每个顶点在预先分配的节点数组中对应一个槽位。节点包含键、父亲、孩子、左右兄弟指针、度数和标记；根表与孩子表均采用循环双向链表，另存最小根指针。插入只将单节点树并入根表；取最小值时提升其孩子，再按度数合并同度树。实现通过摘下当前根再处理的方式遍历根表，避免一边合并一边依赖失效的循环终止点。

```
INSERT(v, key):
  initialize node[v] as a singleton unmarked root
  splice node[v] into root list; update minimum
EXTRACT_MIN():
  remove minimum root z
  promote all children of z to unmarked roots
  while two roots have the same degree:
    link the larger-key root under the smaller-key root
  rebuild root list and minimum; return z
DECREASE(v, key):
  node[v].key = key
  if v violates heap order with its parent:
    cut v to the root list
    mark its parent on the first child loss
    repeatedly cut marked ancestors after another child loss
  update minimum if needed
```

Listing 2: Fibonacci 堆的关键操作

标记记录非根节点是否已失去一个孩子。势函数可取 $\Phi = t + 2q$，其中 t 为根数，q 为标记节点数；级联剪切造成的标记减少可支付剪切代价。Fibonacci 堆由此支持摊还 $O(1)$ 的插入和减键、摊还 $O(\log n)$ 的取最小值，得到 Dijkstra 的 $O(m + n \log n)$ 界 [1]。该界描述总操作序列，并不保证每一次取最小值都只需对数时间。

2.5 单调 radix heap

radix heap 保存上次取出的键 L。键 k 放入编号 $b(k) = \mathrm{bitwidth}(k \operatorname{xor} L)$ 的桶，其中桶零只含等于 L 的键。若桶零为空，从最小非空桶找出最小键作为新 L，并按新 L 重新分配该桶；重新分配后的桶编号会减小。减键使用记录了桶编号和桶内位置的句柄，跨桶移动时交换删除并修复被交换元素的句柄。

```
POP():
  if bucket[0] is empty:
    b = smallest nonempty bucket id
    L = minimum key in bucket[b]
    redistribute every entry in bucket[b] using bitwidth(key XOR L)
  remove and return an entry from bucket[0]
  if all buckets are empty: reset L = 0 for the next query
```

Listing 3: Radix heap 的取最小值

合法插入要求 $k \geq L$，而非要求所有插入键按全局顺序递增。Dijkstra 的非负边权保证新发现候选距离不小于当前取出的有效距离，符合这一约束。当前实现只有 32 个桶，桶内位置句柄占 25 位；它不是任意 64 位键范围上的通用 radix heap。本报告仅针对已通过交叉校验的输入，未来扩大距离或桶容量范围时需先修改实现并验证边界。

2.6 两种标签修正参照算法

FIFO SPFA。队列存 (d, v) 条目；成功松弛后插入新条目，取出后跳过过期距离。它没有单独的在队标记，可能出现重复条目，因此内存和操作次数不应直接套用“一顶点一个队列项”的分析。

SLF+LLL SPFA。双端队列只存顶点，并维护在队标记。成功松弛未在队的顶点时，若其标签小于队首则从前端插入（SLF），否则插到后端。处理队首前，若其标签大于在队顶点的平均标签，则移至队尾（LLL）。通过标签总和维护平均值，队内顶点标签减少时同步更新总和。两条启发规则只调整松弛顺序，不提供 Dijkstra 式的出队定型保证。

```
ENQUEUE(v):
  if queue empty or d[v] < d[front]: push_front(v)
  else: push_back(v)
  in_queue[v] = true; sum += d[v]; count += 1
PROCESS():
  while d[front] * count > sum: rotate front to back
  u = pop_front(); in_queue[u] = false
  sum -= d[u]; count -= 1
  relax outgoing arcs of u
  update sum on an already queued label decrease
  otherwise enqueue each newly improved vertex
```

Listing 4: SLF+LLL 的队列调度规则

第 3 章 测试结果

3.1 环境与计时口径

表 1: 本轮实验环境与配置

项目	配置
平台	同一台 Intel Core i9-14900HX；WSL2 Arch Linux，32 个逻辑 CPU
工具链	GCC 16.2.1；CMake 4.4.3；Ninja 1.13.2；Google Benchmark 1.9.5
编译	C++20，Release，`-O3 -DNDEBUG`
输入与执行	二进制与图数据位于 WSL ext4；输入复制后用 SHA-256 核对
运行策略	benchmark 串行执行；随机图和复杂度各三轮；路网每实现一个完整批次
计时	自动迭代最短目标 0.5 s；路网每次迭代固定包含 1000 查询
正确性	每个 benchmark 计时前执行跨实现距离数组校验

图生成、文本解析、优先队列构造与距离数组初次分配位于计时循环外；每次搜索对距离数组的重新初始化仍在计时范围内。SLF+LLL 的局部在队标记数组分配在其搜索函数中，包含在计时中。随机图与复杂度曲线使用每次调用的 CPU 时间，误差棒为三轮结果的**样本标准差**，不是置信区间，也不是 1000 查询的离散程度。路网表使用搜索循环中墙钟时间除以查询数；原始 CSV 同时保留整批 CPU 与墙钟数据。

3.2 正确性用例表

本轮 CTest 注册 126 项：124 项通过，2 项因不满足 radix 单调插入前提而按设计跳过，没有失败。Release 构建关闭了依赖断言的 Debug death tests，不能将本轮结果解释为这部分测试也已经执行。测试记录保留于运行证据中。

表 2: 测试目的、预期与实际行为

用例类别	目的与预期	实际结果	异常解释	状态
DIMACS 解析	合法输入可解析，坏计数、越界应拒绝	31 项通过	无	通过
堆接口与差分	插入、减键和复用符合模型	适用用例通过	无	通过
Radix 任意键模型	任意键序列违背单调前提	两项主动跳过	模型不适用	跳过
Dijkstra 差分	完整距离等于参照	所有类型通过	无	通过
段树回归	边界叶子与相等键更新正确	五项通过	无	通过
随机图启动检查	多种种子、源点结果一致	校验通过	无	通过
NY/BAY 启动检查	前三查询完整距离一致	两图均通过	仅覆盖前三条	通过

堆单元测试还覆盖 PBDS binomial 和 rc-binomial 类型；它们不在本次七种 Dijkstra 性能实现集合中。路网启动检查的参照为 SLF+LLL，校验对象为七种堆。这与对全部实现、全部查询各自使用独立第三方程序验证有区别。

3.3 USA 路网输入与结果

NY 图有 $n = 264346$、$m = 733846$，BAY 图有 $n = 321270$、$m = 800172$；两图均使用完整的 1000 对查询。每个实现运行一次完整批次，搜索总计 $9 \times 2 \times 1000$ 次。图规模与输入 SHA-256 保留在 `data/provenance.json`；按课程要求，报告目录不附带大型路网原始图文件，只列数据来源链接 [3]。

表 3: 平均每查询墙钟耗时；加速比为标准库耗时除以当前实现耗时

实现	NY/ms	BAY/ms	NY/加速比	BAY/加速比
std PQ	19.849	20.518	1.00	1.00
Binary heap	19.720	20.630	1.01	0.99
Segment tree	36.344	42.261	0.55	0.49
Fibonacci heap	44.803	51.184	0.44	0.40
Radix heap	12.621	13.873	1.57	1.48
PBDS pairing	26.985	30.088	0.74	0.68
PBDS thin	36.818	42.577	0.54	0.48
FIFO SPFA	249.409	353.306	0.08	0.06
SLF+LLL SPFA	7.267	9.258	2.73	2.22

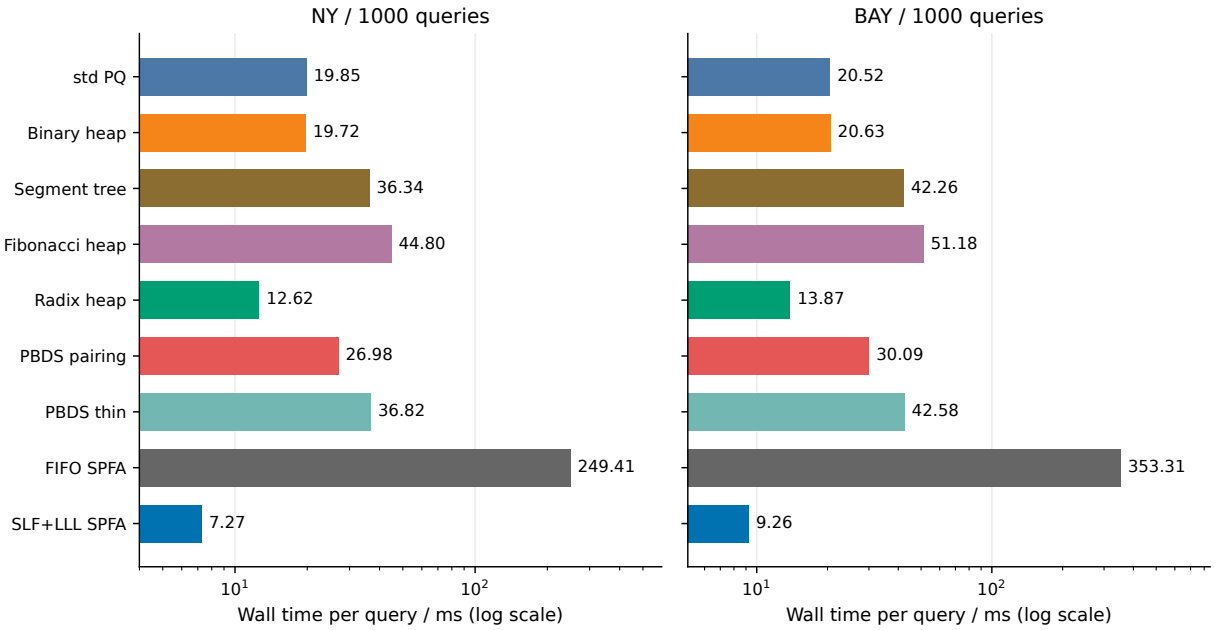


图 1: NY 与 BAY 的平均查询耗时。横轴为对数；每项均为一个完整 1000 查询批次，未估计重复运行方差。

NY/BAY 中 radix 相对标准库的加速比分别约为 1.57、1.48，SLF+LLL 为 2.73、2.22；Fibonacci 的耗时则为标准库的 2.26、2.49 倍。FIFO SPFA 分别需要 249.409、353.306 ms/查询，明显更慢。结论仅针对本轮输入，不能推出 FIFO 在所有图上都慢，或 SLF+LLL 总有优势。

3.4 随机图参数扫描

随机有向图共有 19 组 (n, m)、每组九种实现，合计 171 个配置。允许重边，排除自环，权重范围为 $[0, 200000]$，源点为 1，图不保证连通。同一 C++ 生成器的可达性诊断确认：$n = 163840$ 的两组图 $(m = 163839, 327678)$ 及 $n = 10, m = 9$ 都仅源点自身可达，可达弧数为零。这些结果主要反映初始化与调用开销，不能代表遍历全图的代价。诊断保留在 `data/random-reachability.csv`，原始计时保持不变。

表 4: 三组随机图的平均 CPU 时间，单位 ms；对应弧数依次为 327552、327672、327676

实现	$n = 2560$	$n = 40960$	$n = 81920$
std PQ	1.111	8.050	11.845
Binary heap	0.518	6.046	11.771
Segment tree	0.575	6.468	12.769
Fibonacci heap	0.808	12.751	26.152
Radix heap	0.424	3.246	6.252
PBDS pairing	0.736	9.412	18.288
PBDS thin	1.008	12.700	23.544
FIFO SPFA	0.646	3.710	5.074
SLF+LLL SPFA	0.555	2.960	3.751

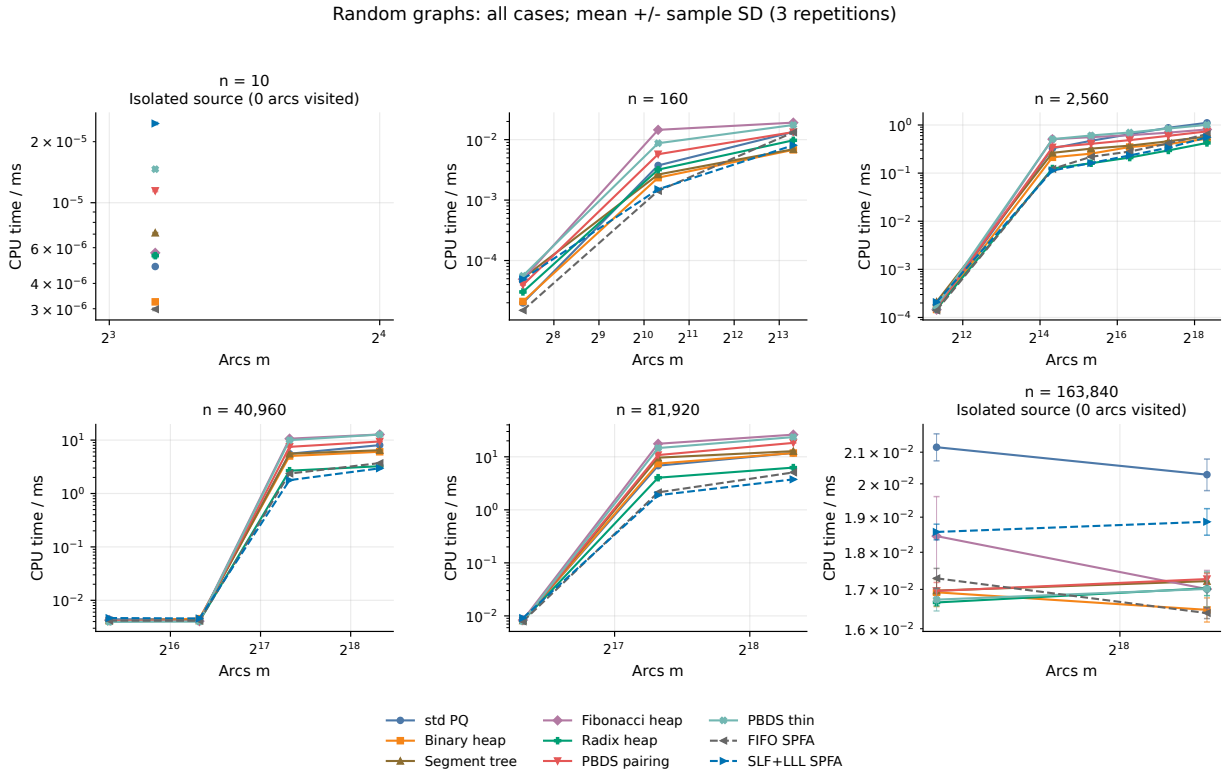


图 2: 随机图全部参数，按固定顶点数分面，只在同一面板内改变弧数。误差棒为三轮样本标准差，纵轴为对数；孤立源点已在面板标题标注。

3.5 固定密度实验与拟合

固定密度实验使用两族图：稀疏族 $m = 4n$，$n = 2^{10}, \ldots, 2^{17}$；稠密族 $m = n(n-1)/4$，$n = 2^7, \ldots, 2^{12}$。每族每个规模都测量九种实现，合计 $9(8+6) = 126$ 组。

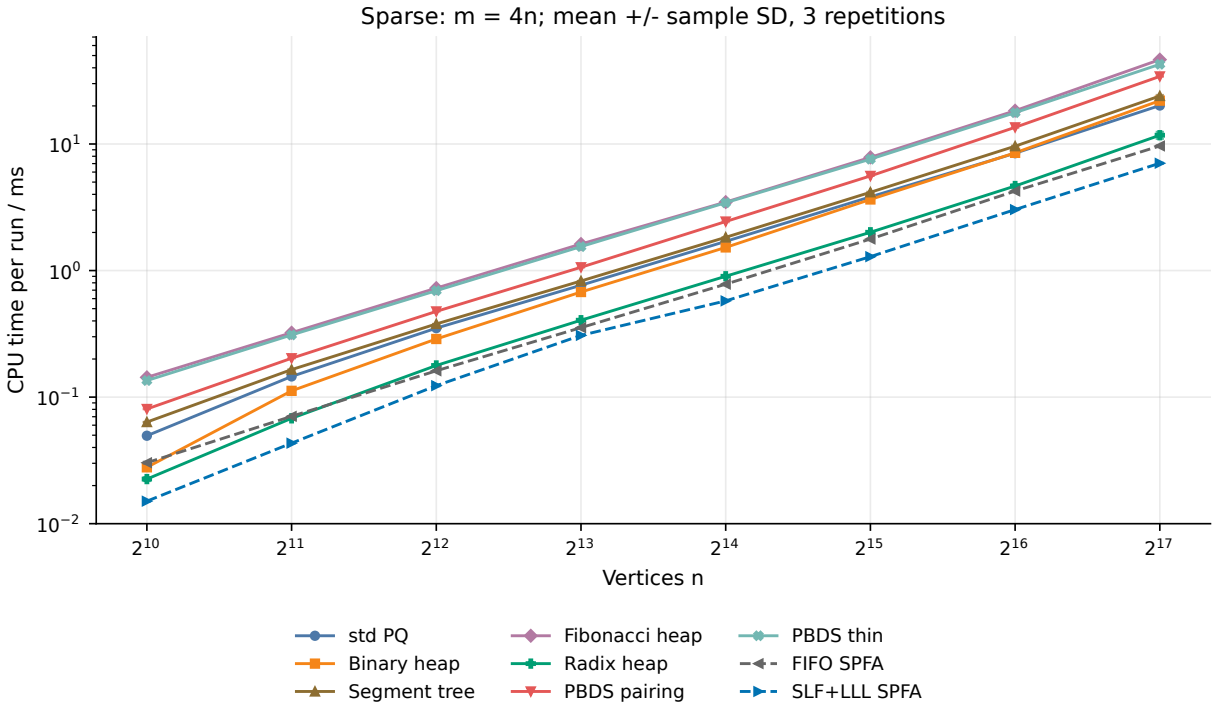


图 3: 稀疏族的平均 CPU 时间，$m = 4n$。误差棒表示三轮样本标准差。

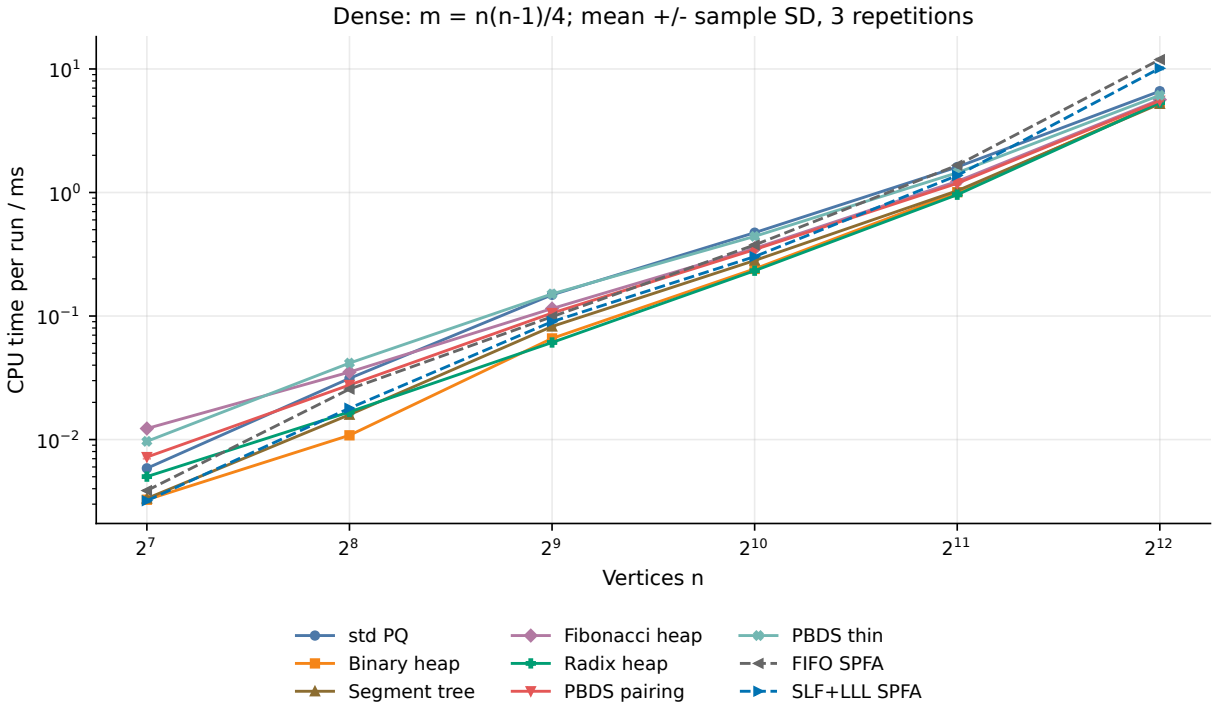


图 4: 稠密族的平均 CPU 时间，$m = n(n-1)/4$。误差棒表示三轮样本标准差。

Google Benchmark 分别固定 $f(n) = n \log_2 n$ 和 $f(n) = n^2$，用过原点的最小二乘形式 $T(n) \approx cf(n)$ 拟合。表5 的稀疏系数单位为 ns/$(n \log_2 n)$，稠密系数为 ns/n^2，两列系数不能直接互相比较大小。RMS 为库定义的归一化残差百分比，衡量有限范围内模型贴合程度。

表 5: 固定模型的 CPU 系数与 RMS；所有值由 CSV 自动导出

实现	稀疏系数	稀疏 RMS	稠密系数	稠密 RMS
std PQ	8.806	9.1%	0.393	8.0%
Binary heap	9.444	16.4%	0.314	14.3%
Segment tree	10.361	13.4%	0.307	10.7%
Fibonacci heap	20.008	14.5%	0.334	5.9%
Radix heap	5.063	16.4%	0.313	14.1%
PBDS pairing	14.729	15.1%	0.327	6.8%
PBDS thin	18.562	10.9%	0.364	3.9%
FIFO SPFA	4.245	8.1%	0.690	23.2%
SLF+LLL SPFA	3.089	8.7%	0.586	24.3%

把相同模型套到 SPFA 只是一种经验参照，不代表证明了其渐近界。稠密图下 Fibonacci 的理论项为 $m + n \log n = \Theta(n^2)$；比较堆 Dijkstra 的常规上界含有 $m \log n$，但有限规模内扫描边的代价也可能主导计时，不能仅凭 n^2 拟合好就证明去掉了对数因子。

第 4 章 分析与讨论

4.1 时间与空间复杂度

表6 讨论从给定源点进行一次完整搜索。图本身占 $O(n+m)$ 空间，距离数组占 $O(n)$；表中的队列空间单列。标准库版本使用懒删除，其堆中可能保留 $O(m)$ 个条目，与每个顶点仅一个在堆条目的减键实现存在实质差别。

表 6: 队列策略的代价与适用条件

策略	时间上界或分析说明	适用条件	队列空间
标准库懒删除	$O(n+m\log(m+1))$	非负权；可重复条目	$O(m+1)$
索引二叉堆	$O((n+m)\log(n+1))$	非负权；可直接减键	$O(n)$
段树堆	$O((n+m)\log(n+1))$	顶点固定映射为叶子	$O(n)$
Fibonacci 堆	$O(m+n\log(n+1))$，摊还	非负权；直接定位节点	$O(n)$
PBDS 两种堆	依库堆的具体摊还界，不在本项目中证明	GCC 扩展与有效句柄	$O(n)$
Radix heap	按键位宽 B 计为 $O(m+nB)$ 的桶迁移分析	非负整数；单调键与容量限制	$O(n+B)$
FIFO SPFA	与成功松弛和重复入队有关，存在坏例	本实验只使用非负权	依实际条目数
SLF+LLL	启发式不能保证 Dijkstra 式效率	本实验只使用非负权	$O(n)$

对 Fibonacci 堆，所有弧扫描和成功减键总计 $O(m)$，每个顶点至多一次有效取最小值，合计 $O(n\log n)$。稀疏图上该界与二叉堆同阶，但前者每次操作涉及多条链接维护，理论差异不保证常数因子更小。Radix heap 的分析利用桶重分配时编号降低；若最大有效键界为 U，可令 $B=\lceil\log_2(U+1)\rceil$。文献中的专门 radix 设计可给出 $O(m+n\log C)$ 的 Dijkstra 界（C 为边权上界）[2]；本项目没有证明其有限 32 桶和压缩句柄实现满足文献全部设定，因此不直接照搬更紧的界。

本轮没有采集各算法的峰值内存曲线、硬件缓存计数、成功松弛次数或桶重分配次数。空间复杂度分析来自数据结构，不能当作已经完成的内存实测。

4.2 为何 Fibonacci 堆没有最快

本轮 NY、BAY 的平均出度只有约 2.78 和 2.49，属于稀疏路网。Fibonacci 堆减少了减键的渐近代价，但仍须执行合并同度根、维护链表和级联剪切。其节点虽然一次性分配在数组中，操作依然沿节点链接跳转；二叉堆的大部分状态则是紧凑数组。较多的链接操作及访问模式可能解释部分差距，这是基于实现结构的解释，尚需性能计数器与操作计数实验验证，不能把缓存未命中当作本轮已经观察到的事实。

稠密族中边扫描工作随 n^2 增长，堆维护占比可能下降，因此某些队列的差距缩小。本轮最大稠密规模为 $n=4096$，不存在“在足够大规模最终谁胜出”的实验结论。随机图的最大顶点规模虽较大，但它属于另一种稀疏度，不能拿来代替大规模稠密实验。

4.3 Radix 与 SPFA 结果的意义

在非负整数路网上，radix heap 利用键的单调性，减少比较堆中的上浮和下沉工作，成为本轮七种 Dijkstra 队列中最快的一种。它仍要求正确的键范围与句柄容量，不能直接替换一般实数键或非单调任务中的优先队列。

SLF+LLL 在两图上比所有堆版本更快，FIFO 版本却慢一个数量级以上。这说明标签修正算法高度依赖调度顺序，不能只以“都是 SPFA”推断性能。两种实现还分别使用重复条目与在队标记，故差距也不能只归因于 SLF/LLL 一个因素。随机图上的良好表现不构成对抗输入下的效率保证。

4.4 模型拟合与测量限制

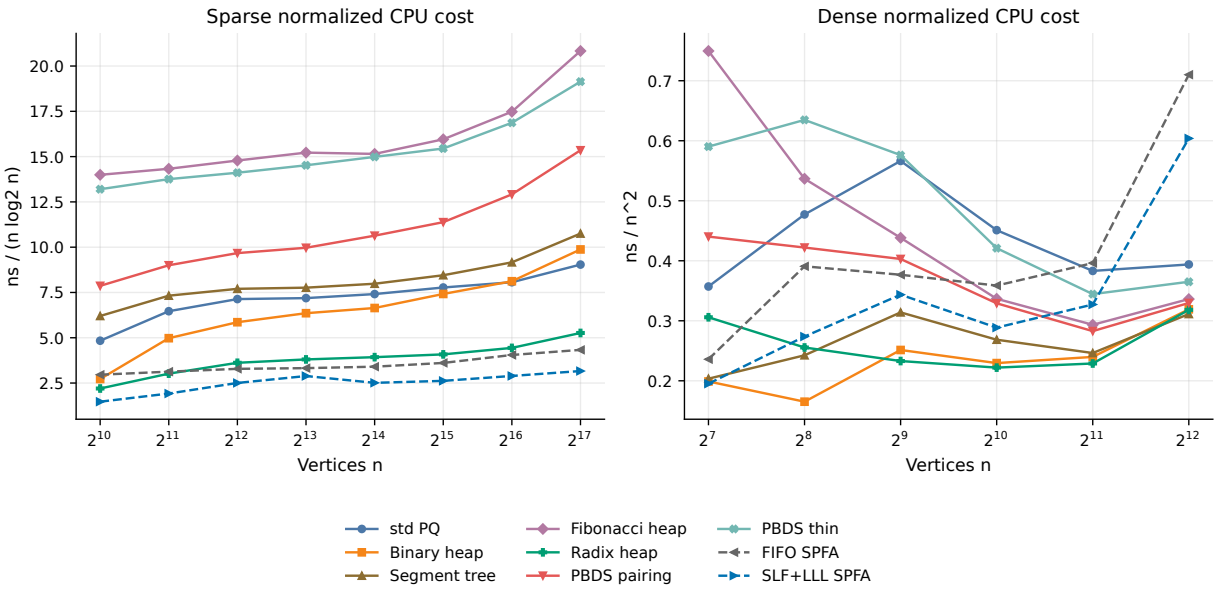


图 5: 按预设模型归一化的 CPU 时间。曲线并非严格常数；该图用于观察模型偏差与有限规模效应，不用于证明复杂度。

本轮复杂度图是有限个离散规模上的计时。归一化曲线的变化既可能来自算法操作数变化，也可能来自缓存、编译器、频率或输入可达性变化。RMS 小只能说明所选范围内模型贴合，不能排除另一种模型或推出紧确界。

三轮随机/复杂度实验不足以稳定估计尾部分布；路网只有单批次、无逐查询分布，两个路网规模也不足以作可靠的规模拟合。WSL 共享宿主硬件，本轮未固定 CPU 亲和性或测量宿主负载；绝对时间需复验。多实现对拍和单元测试仍不等于形式化证明。

4.5 后续改进与底稿结论

后续应增加路网独立批次与逐查询记录，扩大经校验的图规模；记录松弛、过期条目、合并与桶迁移次数，并用性能计数器检验常数开销的解释。若研究真正的点对点查询，应统一加入目标提前终止，再考虑双向搜索或预处理算法。

本轮结果说明，理论摊还优势与实际运行速度需要分别评价，实测结论也应受输入范围约束。

附录 A 源代码与复现说明

A.1 源码组织

源码位于项目的 pq-test/ 下。以下给出与本报告对应的文件，完整源码是否需要逐页附入报告，以当前课程要求为准。

表 7: 主要源码与功能对应

文件	功能
csr-graph.hpp、dimacs-parser.hpp	共享图结构与文件解析
general-dijkstra.hpp	共用 Dijkstra 主循环
heap-adaptor.hpp	时间戳接口、标准库与 PBDS 适配器
binary-heap.hpp、segment-tree-heap.hpp	二叉堆与段树
fibonacci-heap.hpp、radix-heap.hpp	Fibonacci 与 radix heap
general-bellman-ford.hpp	两种标签修正算法
dij-bench-v2.cpp	随机图参数扫描
dij-bench-complexity.cpp	固定密度复杂度实验
dij-bench-dimacs.cpp	路网查询批次基准
heap-adaptor-test.cpp、dimacs-parser-test.cpp	接口、差分、回归及解析测试

下列节选保留实际主循环逻辑，省略文件头中的身份信息，并补充解释性注释。它用于说明共享接口，不替代完整源码提交。

```cpp
// Shared Dijkstra loop, excerpted without identifying file-header comments.
// The caller allocates the distance vector and supplies a reusable empty heap.
template <typename Heap>
inline void dijkstra(const Graph &g, uint32_t s,
                     std::vector<int64_t> &dis, Heap &heap) {
  // Leave headroom for distance addition on the validated input domain.
  constexpr int64_t INF = std::numeric_limits<int64_t>::max() / 4;
  // Initialization is part of every timed search.
  std::fill(dis.begin(), dis.end(), INF);
  heap.push_or_decrease(0, s);
  dis[s] = 0;
  while (!heap.empty()) {
    auto [d, u] = heap.pop();
    // Standard-library lazy insertion can leave outdated entries.
    if (d > dis[u]) continue;
    // CSR adjacency traversal is shared by all heap variants.
    for (auto [v, w] : g[u]) {
      auto nd = dis[u] + w;
      if (dis[v] > nd) {
        dis[v] = nd;
        // Insert on first discovery, decrease an existing key otherwise.
        heap.push_or_decrease(dis[v], v);
      }
    }
  }
  // This implementation deliberately drains the heap, even for p2p input.
}
```

Listing 5: 共享主循环节选

待补充：课程结构要求规定提交源码至少 30% 的行带有注释。此底稿没有声称项目全部源码已满足该比例。正式提交前需按课程口径审查实际提交的源码，并补全必要注释；不能用本页注释比例替代整体审查。

A.2 数据与作图复现

`reports/data/` 保留 CSV 快照、测量校验结果及经过匿名化处理的溯源信息。

总表 `all-results.csv` 含 2151 行；计时汇总 `timing-summary.csv` 含 315 组配置。脚本从 `run_type=iteration` 的 909 条原始测量行重新计算均值和标准差，不会把均值、CV 或 RMS 行当作新样本。

在项目根目录执行：

```
python -m pip install -r tools/requirements-plot.txt
python tools/plot_benchmarks.py
python reports/build_report.py
```

各作图脚本也可独立运行，并通过 `--data-dir`、`--output-dir` 指定目录。图同时保存为矢量 PDF 和 PNG；表格由 `generate_report_tables.py` 生成到 `reports/tables/`，不应手工改动数值。

从源码重新执行 benchmark 的基本步骤如下；不同机器请自行指定构建目录和路网数据目录。

```
cmake -S . -B build-wsl -G Ninja -DCMAKE_BUILD_TYPE=Release
cmake --build build-wsl --target heap-adaptor-test dimacs-parser-test \
  dij-bench-v2 dij-bench-complexity dij-bench-dimacs
ctest --test-dir build-wsl --output-on-failure
build-wsl/pq-test/dij-bench-v2 --benchmark_min_time=0.5s \
  --benchmark_repetitions=3 --benchmark_out=random.json \
  --benchmark_out_format=json
build-wsl/pq-test/dij-bench-complexity --benchmark_min_time=0.5s \
  --benchmark_repetitions=3 --benchmark_out=complexity.json \
  --benchmark_out_format=json
build-wsl/pq-test/dij-bench-dimacs --gr DATA/USA-road-d.NY.gr \
  --benchmark_repetitions=1 --benchmark_out=ny.json \
  --benchmark_out_format=json
build-wsl/pq-test/dij-bench-dimacs --gr DATA/USA-road-d.BAY.gr \
  --benchmark_repetitions=1 --benchmark_out=bay.json \
  --benchmark_out_format=json
```

本地原始运行目录另保留从 JSON 导出 CSV 的脚本和完整命令。若采用新的参数或查询集，应保留新的溯源文件并重新生成图表；本报告数据快照不会自动替换。

参考文献

[1] M. L. Fredman, R. E. Tarjan. “Fibonacci heaps and their uses in improved network optimization algorithms.” *Journal of the ACM*, 34(3), pp. 596–615, 1987. https://doi.org/10.1145/28869.28874.

[2] R. K. Ahuja, K. Mehlhorn, J. B. Orlin, R. E. Tarjan. “Faster algorithms for the shortest path problem.” *Journal of the ACM*, 37(2), pp. 213–223, 1990. https://doi.org/10.1145/77600.77615.

[3] C. Demetrescu, A. V. Goldberg, D. S. Johnson. *9th DIMACS Implementation Challenge: Shortest Paths*. 官方数据目录：http://www.diag.uniroma1.it/challenge9/data/USA-road-d/. NY/BAY 的原始文件名分别为 `USA-road-d.NY.gr.gz` 与 `USA-road-d.BAY.gr.gz`。本项目的数据来源与归档校验见 `docs/DATA.md`。

[4] Google. *Google Benchmark User Guide*, vendored version 1.9.5. https://github.com/google/benchmark/blob/v1.9.5/docs/user_guide.md. 本项目拟合及统计字段同时可在本地库实现中核查。

[5] Free Software Foundation. *Policy-Based Data Structures, libstdc++ Manual*. https://gcc.gnu.org/onlinedocs/libstdc++/manual/policy_data_structures.html.

作者分工

本版本用于匿名审阅。下列代号仅为待填写的分工槽位，不代表已经确定的实际贡献。

作者代号	实际完成的工作	核对状态
作者 A	待填写实际算法与数据结构实现工作	待本人确认
作者 B	待填写实际测试、数据及作图工作	待本人确认
作者 C	待填写实际分析、报告及复现工作	待本人确认

待补充：按实际人数增删行。根据课程规定如实补充外部代码、工具辅助及引用的说明，不将模板预设分工当作已发生事实。

声明

下文为供全体作者审阅的声明草稿，尚未签署：

本组确认已审阅题为《不同优先队列下 Dijkstra 算法的实现与评测》的项目报告，将按实际贡献确认署名，并如实列明参考资料、使用的库与工具辅助。有关独立完成要求及课程允许使用的工具，须由全体作者依据课程规定核实后签署。

待补充：根据实际情况与课程原文确定最终声明。不得在未确认时把本段作为已签署的独立完成承诺。

签名

匿名审阅版本不包含签名或身份标识。最终交给助教的版本按课程要求补签。